REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Vicente Silva Rios

9 de abril de 2026

01:35

1. Entorno experimental

1.1. Infraestructura Experimental

Para asegurar que el experimento sea repetible a continuación se detalla el entorno de hardware y software en el cual se ejecutaron las pruebas, la tarea fue realizada en una computadora con las siguientes características de hardware:

- **Procesador:** Intel Core i5-12400F, 2.5 GHz
- **Memoria RAM:** 32 GB DDR4.
- **Almacenamiento:** SSD de 1 TB.
- **Sistema Operativo Host:** Windows 11 Pro.

Para la compilación y ejecución de los programas se utilizó el Subsistema de Windows para Linux (WSL), con el siguiente entorno:

- **Distribución:** Ubuntu 24.04.1 LTS.
- **Compilador:** g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0.

En todo momento del funcionamiento del programa la computadora estuvo dedicada solo a eso, con Visual Studio Code y la consola abierta corriendo los algoritmos.

2. Resultados y Análisis

2.0.1. Multiplicación de matrices

En la Figura se compara el tiempo de ejecución entre los algoritmos naive y Strassen para multiplicar matrices cuadradas hasta $N = 256$, se saltaron las matrices con $N = 1024$ debido a excesivo tiempo de ejecucion y hacer mas rapido la tarea, esto tambien en los algoritmos de ordenamiento, A medida que las matrices crecen, ambos aumentan su tiempo de acuerdo con sus complejidades teóricas. Aunque el algoritmo naive tiene una complejidad de $O(n^3)$, resulta más rápido en estas instancias evaluadas. Por su parte, Strassen reduce la complejidad a $O(n^{2,81})$, pero añade un costo extra debido a sus operaciones recursivas y al uso de memoria intermedia. Por lo tanto, para matrices pequeñas y medianas, la estrategia tradicional es más eficiente en la práctica.

Cabe destacar que estos datos se obtuvieron de forma automática. Los archivos `.csv` están disponibles en el repositorio del proyecto para que los resultados puedan ser verificados o reproducidos fácilmente.

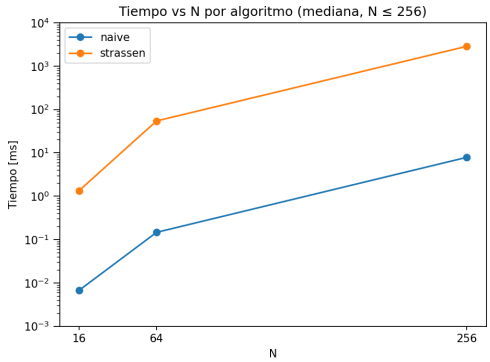

Figura 1: Gráfica de Tiempo vs N entre algoritmos.

2.0.2. Efecto del tipo de matriz en el tiempo de ejecución

Las Figuras muestran cómo varía el tiempo según el tipo de matriz (densa, diagonal o dispersa) y sus dominios D0 y D10.

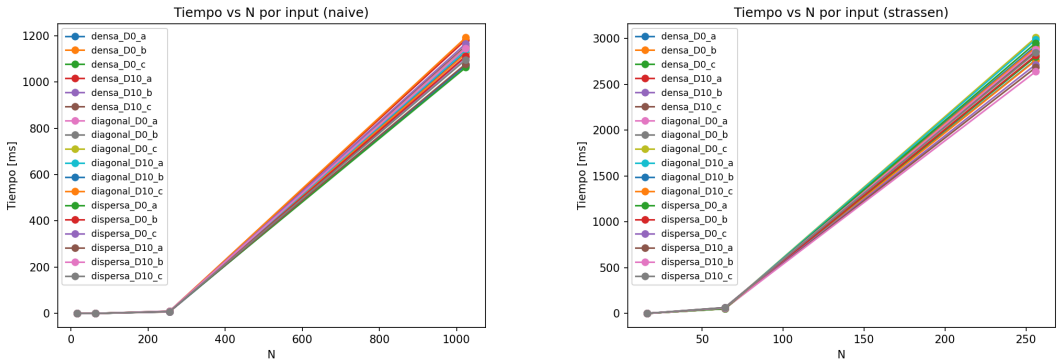

Figura 2: Tiempo vs N por input de cada algoritmo.

El algoritmo naive tarda menos con matrices dispersas y luego con las diagonales, ya que evita realizar cálculos innecesarios con los ceros, reduciendo así la cantidad efectiva de multiplicaciones y sumas. En contraste, Strassen mantiene tiempos muy similares sin importar el tipo de matriz. Esto ocurre porque sus divisiones recursivas ejecutan secuencias fijas de operaciones que no logran aprovechar la presencia de ceros. En síntesis, naive saca ventaja de estructuras simples o vacías, mientras que Strassen es insensible a este factor.

2.0.3. Memoria estimada en algoritmos de matrices

La Figura ilustra la memoria estimada para instancias de hasta 256×256. Se observa que Strassen consume constantemente más memoria que el método naive, lo cual coincide con las estimaciones teóricas:

$$M_{\text{naive}} \approx 3 \cdot n^2 \cdot \texttt{sizeof(int)}, \quad M_{\text{strassen}} \approx 8 \cdot n^2 \cdot \texttt{sizeof(int)}.$$

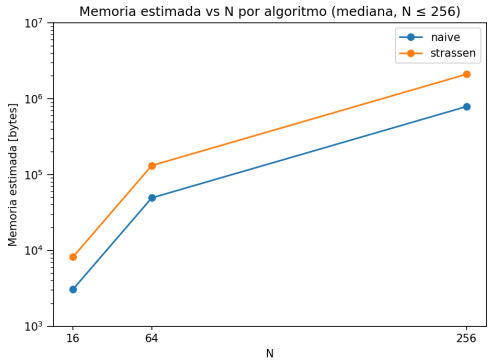

Figura 3: Gráfica de Memoria estimada en función de N para los 2 algoritmos.

Esta diferencia radica en que naive opera directamente sobre las matrices originales y la resultante, mientras que Strassen exige generar múltiples submatrices en cada paso recursivo. Como conclusión, para tamaños moderados, Strassen resulta ser más lento y más demandante en memoria, limitando su atractivo frente a la versión tradicional.

2.0.4. Algoritmos de ordenamiento de arreglos según el input

En las 3 figuras se presentan los tiempos de ejecución de los algoritmos (merge sort, quick sort y std:sort) para distintos tamaños n y tipos de entrada (ascendente, descendente y aleatorio).

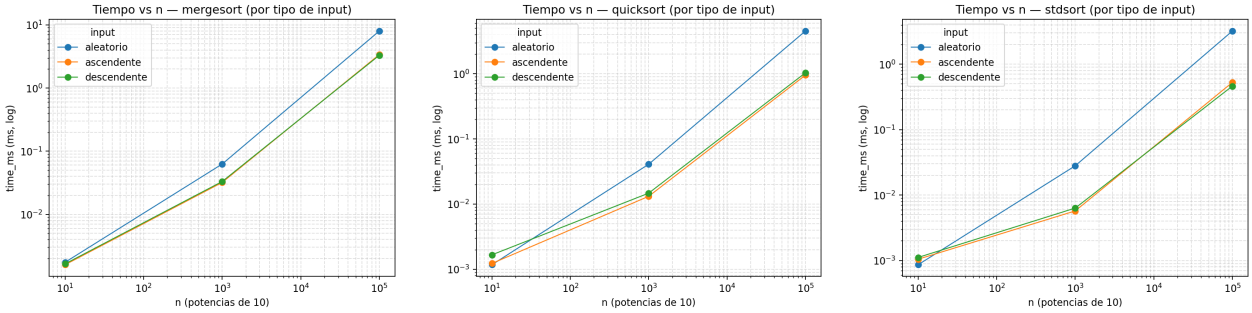

Figura 4: Tiempo vs N por input de cada algoritmo.

Merge sort, fiel a su complejidad teórica de $O(n \log n)$, rinde igual en cualquier caso, mostrando solo un levísimo aumento de tiempo en los escenarios aleatorios.

Quick sort, al utilizar un pivote central, es más lento en arreglos aleatorios debido a que los arreglos ya ordenados le generan particiones mucho más balanceadas. En resumen, los algoritmos cuadráticos dependen fuertemente del orden original de los datos, mientras que los más eficientes mantienen un rendimiento estable.

std::sort, fiel a su complejidad teórica de $O(n \log n)$, garantiza eficiencia en todos los escenarios, pero muestra un tiempo de ejecución notablemente mayor frente a los arreglos aleatorios, al estar altamente optimizado para detectar y aprovechar secuencias existentes, procesa los arreglos preordenados en tiempos casi idénticos y mucho más rápidos al minimizar drásticamente los intercambios en memoria.

2.0.5. Memoria estimada en algoritmos de ordenamiento

En la Figura se expone el uso máximo de memoria extra en función del tamaño del arreglo n para los algoritmos evaluados.

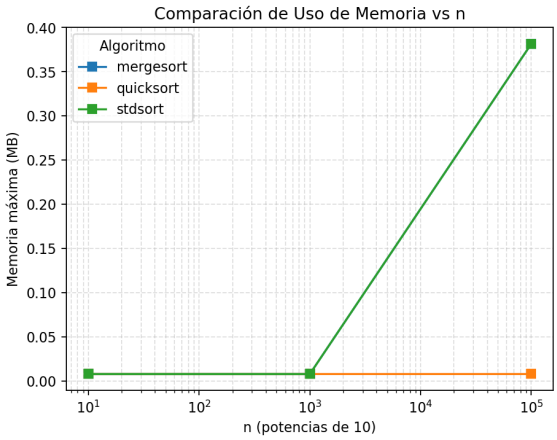

Figura 5: Comparación de uso de memoria vs n entre algoritmos de ordenamiento.

Se evidencia que quicksort mantiene un consumo de memoria extra prácticamente nulo en todos los casos, lo que confirma experimentalmente su naturaleza de ordenamiento.

Por el contrario, tanto mergesort como stdsort muestran un crecimiento lineal que se superpone perfectamente en el gráfico, alcanzando exactamente 400,000 bytes para los arreglos de 10^5 elementos.

En el caso de mergesort, este aumento concuerda con su complejidad espacial teórica de $O(n)$, ya que el algoritmo requiere instanciar arreglos temporales para poder fusionar las mitades.

3. Conclusiones

El estudio demostró que la eficiencia de un algoritmo no se reduce únicamente a su complejidad asintótica. Como revelan los resultados, factores como el tamaño de las instancias, la estructura de los datos iniciales y el consumo de memoria resultan decisivos para el rendimiento real. Específicamente, en la multiplicación de matrices, el algoritmo tradicional (naive) resultó ser más rápido y eficiente en memoria que Strassen para instancias de hasta $N = 256$, ya que este último añade un gran costo por sus operaciones recursivas. Por otro lado, en los algoritmos de ordenamiento, se observó que mergesort mantiene un rendimiento estable sin importar la entrada a costa de un mayor uso de memoria, mientras que quicksort es más sensible al orden inicial de los datos (siendo más lento en arreglos aleatorios) pero no consume memoria extra, y std::sort supera ampliamente a ambos. Aunque la teoría brinda una base indispensable, es la experimentación la que expone las verdaderas ventajas y limitaciones en la práctica.De este modo, se logra comprender el comportamiento algorítmico más allá del análisis matemático.

Seleccionar el algoritmo adecuado exige equilibrar las predicciones teóricas con la evidencia empírica, lo que en definitiva forja un criterio mucho más crítico y fundamentado en el desarrollo computacional.

4. Apéndice 1

Aqui adjunto links varios a paginas que use para realizar la tarea informandome de ciertos algoritmos, ademas de que para casos en los cuales donde no encontre la informacion tan rapido me apoye del uso de LLM's tales sean como Gemini o Claude.

4.0.1. Referencias de codigo

1. GeeksforGeeks: Merge Sort
2. GeeksforGeeks: Matrix Multiplication in C++
3. GeeksforGeeks: Strassen's Matrix Multiplication
4. GeeksforGeeks: Quick Sort Algorithm
5. GeeksforGeeks: Quick Sort Hoare (Utilizado)Algorithm